Jayesh Sharma



Full Stack Engineer

- Full Stack Engineer with 2+ years of experience in React.js, Node.js, cloud deployment, and AI-integrated systems, along with AWS Certified Cloud Practitioner certification.
- Specialises in developing complex Frontend components while maintaining re-usability, minimum re-renders and seamless integrations with the backend.
- Solid expertise in AI integration into the workflow with relevant experience in developing, deploying, and automating complete integrated frontend and backend systems incorporating Cloud Systems/VPS, Containerization, and automating CI/CD Pipelines using GitHub Actions.
- Built deeply nested complex backend logic and microservices end-to-end to support core functionalities and seamless integration.
- Aspires to leverage technical skills and multidisciplinary project experience to enhance web application development, deployment, and automation in a fast-paced environment.
- Skilled in RESTful API integration and third-party services.
- Proven ability to lead feature development, collaborate with cross-functional teams, and deliver projects on time.
- Alongside technical proficiency, I bring a creative mindset, strong organisational skills, and a proven record of meeting deadlines, making me well-suited for multifaceted roles that blend technology, teamwork, and cross-functional collaboration.
- Familiar with VPS and Cloud deployment, setting up automated CI/CDs, hosting and maintaining Databases.
- Proficient in Git-based version control systems, Agile practices, and collaborative tools like JIRA.
- Strong debugging skills, an eye for attention to detail, to identify and address crucial problems.
- Excellent interpersonal and communication skills, with the ability to understand functional requirements and deliver tailored web solutions.

Technical Skills:

Frontend: React.js, TypeScript, Zustand, Redux, Tailwind CSS, Vite
Backend: Node.js, Express.js, FastAPI
Databases: MongoDB, MySQL, PostgreSQL
Cloud & DevOps: AWS, VPS, Docker, GitHub Actions
AI/ML: OpenAI API, Ollama
Tools: Git

EXPERIENCE

AdlerQA Technologies Pvt. Ltd., Indore *- Full Stack Engineer*

May 2024 to Present

Responsibilities

- Working in a super fast-paced startup environment equipped me with some of the most crucial skills, such as delivering under pressure while maintaining the integrity of the product. Always saying yes to things I had no idea how to accomplish, which I consistently managed to complete with precision. I believe saying yes to these challenges forged me into a great adapter and finally gave my potential a chance to showcase itself.
- Independently developed multiple end-to-end functional services, working prototypes, and even products.
- Co-led a team of 8 developers with the backend head to develop and deliver products well before the timeline.

- Solely took care of the setup, security and handling of all automated processes, VPS Servers and cloud services running for the company.
- Implemented user onboarding workflows, an Application tracking system or ATS and face/speech/object recognition systems for proctoring, all in Python.
- Utilised React, modern JavaScript, and REST APIs to create responsive and dynamic user interfaces.
- Built efficient state management systems to ensure smooth interaction and data synchronisation across components.
- Presented a fully functional working prototype to automatically create and run the test scripts, and output the results and observations for any web application using Playwright's MCP AI Agent.
- Actively solved challenges in an always-moving environment, enhancing adaptability and problem-solving skills.
- Gained hands-on expertise in frontend architecture design, collaborative communication, and delivering scalable solutions under tight deadlines.
- I wasn't familiar with most of the skills mentioned above (except for the Frontend ones), I learnt them all on the fly when I was given and trusted with the tasks. I feel that these opportunities made me proficient in them, and more than that, they exposed my potential to me

TECHNICAL PROJECTS

1. Talenteq – AI-powered job portal for candidates and vendors

Project Description:

A web platform where vendors post jobs and get an AI-generated 'Smart List' that scores each candidate against the role. Vendors can upload their own bench, share it with vendor connections, and shortlist applicants through a secure screening round by simply sharing test links. For this project I worked beyond my usual frontend role — handling database management, VPS security, CI/CD, and containerization, all within a short time frame while learning on the fly.

Project Overview:

- **User Interface**: Role-based Access systems for Vendors and Admin. Relevant Dashboards for either of them with Graphs and Charts of engagements.
- **Dedicated Pages** to view applicants, peers' bench list, exclusive private jobs, Data collected through any given job, shortlisted candidates, Test scores, etc.
- **AI-powered matching System**: Consumes the Job's description, location, budget, and other relevant data, takes it, and compares it with the applicant's current location, availability, and keywords found on the data they shared, to output a concise list with the most apt and practical recommendations

Skills Utilised:

- React.js, Redux, Tailwind CSS, Vite
- Node.js, Express.js, MongoDB
- VPS Security and Management
- GitHub Actions
- JWT-based admin/vendor login
- OpenAI API

Impact:

- Automated Data consumption to AI
- Collections of large data pools that contain important information about active members of the QA community - through this pool, we can derive hot topics/skills/tools in the market.
- Minimalist list with most optimisation
- Gave vendors a platform to network with other vendors, share and receive bench resources/requirements

2. Microservice - AI-powered Test Generator and proctoring platform

I was trusted to develop this concept from scratch, on my own. It's a microservice built as an extension of Talenteq — an independent platform that generates tests, maps jobs to their tests and scores, and runs a secure screening round while capturing evidence (audio clips and webcam snapshots), all in its own backend and database. With a few API calls, any project can be added to recognised users and start using the microservice almost instantly.

Project Overview:

- Role-Based Access to the data: Uses token-based Authorization to ensure any request is only served to the rightful viewer.
- Vendor or Admin can perform CRUD operations on the scores, evidence, or any relevant data as long as they own the Job.
- Uses YOLO to detect multiple faces, phones/tablets/laptops, books.
- WebRTC VAD, librosa, and pyannote.audio were used to detect any irregularities in audio frequencies, and flag them when they did.
- Evidence and all other data are encrypted and stored securely on an encrypted AWS S3 bucket.

Skills Utilised:

- React.js, Tailwind CSS, Vite
- Node.js, MongoDB, Python
- Docker
- AWS S3, AWS IAM, VPS
- GitHub Actions
- YOLO, pyannote, FastAPI, webrtcvad
- JWT-based admin/vendor access
- OpenAI API
- Cross-platform Integration

Impact:

- Automated Test generation, Test taking, and proctoring platform capable of achieving tasks without any manual interaction
- A platform for vendors to screen their bench for any given role
- Low graphics compatible- Minimal load on the system while conducting the test

3. ReconcileAI – AI-powered payment reconciliation platform for cash-pay medical & wellness clinics

Project Description:

A full-stack financial reconciliation tool that lets cash-pay clinics upload their EMR bookings and payment-gateway charges as CSVs and automatically matches every payment to its booking using an ML model, replacing hours of manual spreadsheet cross-checking. Each booking-to-payment pair is scored by a dedicated ML microservice — confident matches reconcile automatically, while ambiguous ones surface as review tasks. I owned the entire backend from scratch (data model, async APIs, auth, ML integration, and AWS deployment) and built the React screens on the existing design system.

Project Overview:

- **CSV Ingestion & Mapping**: Users upload EMR-booking and gateway exports; rows are staged, column mappings confirmed, and normalised into Postgres before matching.
- **ML Matching Engine**: A stateless LightGBM scoring service (/score, /train); the backend builds candidate-pair features as Postgres views and writes back reconciled records with confidence scores.
- **Exception Review**: Low-confidence or unmatched pairs become review tasks with AI-generated suggestions, so operators only handle the edge cases.
- **Auth & Multi-tenant Access**: JWT access/refresh authentication scoped per business, so each clinic only ever sees its own financial data.

Skills Utilised:

- React 19, TypeScript, Tailwind CSS v4, Vite, TanStack Query
- FastAPI, Python 3.11, SQLAlchemy 2.0 (async), Alembic
- PostgreSQL 16, pgvector, LightGBM scoring microservice
- AWS EC2, RDS, S3, IAM; nginx, gunicorn/uvicorn, certbot (TLS)
- Docker, Railway, JWT authentication, OpenAI API, httpx

Impact:

- Replaced manual spreadsheet reconciliation with an automated ML pipeline that matches bookings to charges in seconds.
- Focused human effort only on flagged exceptions rather than every transaction, cutting review time dramatically.
- Cleanly separated the stateless ML service from the app, so the model can be retrained and redeployed independently.
- Delivered a production-ready AWS deployment end-to-end (EC2 + nginx + RDS + S3, HTTPS via certbot).

4. WardenIQ – self-hosted AI test-intelligence platform

Project Description:

An open-source, self-hosted test-intelligence platform that turns requirement documents (PRD/HLD/LLD as PDF, DOCX, or Markdown) into a normalized, reusable test suite using an LLM, stores it as a RAG knowledge base, and maps GitHub/GitLab code changes back to the test cases they impact or cover. Built as a single FastAPI service with a no-build vanilla-JS UI, backed by a MongoDB replica set and mongot for vector + text search, with pluggable LLM/embedding providers (Ollama by default, or OpenAI/Anthropic/Mistral) — designed to run entirely on-prem so no data ever leaves the customer's environment.

Project Overview:

- **Test generation from docs**: Produces Functional, E2E, API, and Non-functional test cases from uploaded requirements, with coverage-depth and focus controls; steps and cases are normalized and reusable, and embeddings dedupe similar cases.
- **Versioning & diffing**: Features are versioned (group_id + version); diffing keeps still-valid cases, retires obsolete ones, and adds new ones on each version.
- **Code-to-test mapping**: GitHub/GitLab integration polls PRs and commits, maps PRs to features, runs LLM-judged coverage review, and performs per-repo/branch impact analysis.
- **Mind Map (deep code analysis)**: Fetches each repo at a chosen branch, chunks and embeds its source into a vector store, then has the LLM judge every feature's cases as covered / partial / uncovered with rationale and cited files, using hybrid retrieval.
- Also supports release-regression test cycles, a "Start Developing" flow that generates feature code and opens a PR, Jira integration, and passwordless OTP auth with encrypted key storage.

Skills Utilised:

- Python, FastAPI
- MongoDB (Percona Server 8.3, 3-node replica set), mongot vector + text search (RAG, embeddings, hybrid retrieval)
- Ollama (nomic-embed-text, qwen2.5); pluggable LLM (OpenAI / Anthropic / Mistral)
- GitHub / GitLab REST APIs, Jira integration
- Docker, Docker Compose, vanilla-JS single-file UI
- Fernet encryption, OTP-based auth

Impact:

- Turns raw requirement docs into a reusable, deduplicated test suite automatically, cutting manual test-authoring effort.
- Continuously maps code changes back to impacted test cases, so coverage gaps surface on every PR.

- Fully on-prem and open-source — no data leaves the customer's environment, and LLM/embedding providers are swappable.

5. ArchitonX – AI-native construction management SaaS platform

Project Description:

A full-stack, multi-tenant platform for running residential construction projects end-to-end — documents, drawings, BIM, BOQ, tenders, contracts, materials, site diary, inspections, NCR/defects, RFIs, variations, payments, and reporting — inside workspaces with role-based access. I work across the entire stack, from React frontends and Express APIs to a PostgreSQL/Drizzle data layer, and own the platform's authorization architecture — moving it from fixed-role RBAC toward a fine-grained, relationship-based (ReBAC) permission engine — while wiring Google Gemini into the workflow for document intelligence, site-photo analysis, and project insights.

Project Overview:

- **Multi-tenant Workspaces & Access Control**: Workspace-scoped, role-based access that ties every project, module, and route to the right user; I led the shift from fixed roles to a relationship-based engine for finer, seat-aware authorization and strict tenant isolation.
- **Construction Modules**: End-to-end modules for documents & drawings, BIM, BOQ, tenders & contracts, materials, site diary, inspections, NCR/defects, RFIs, variations, payments, and reports.
- **AI Layer**: Google Gemini powering an in-app assistant, document intelligence, site-photo analysis, and risk / schedule / procurement insights.
- **Billing & Integrations**: Stripe billing with plan tiers and upgrade gating, Resend transactional emails, Sentry and PostHog monitoring, object storage, OpenCDE, and a plugin system.

Skills Utilised:

- React 18, TypeScript, Vite, TanStack Query, Tailwind CSS, shadcn/Radix UI
- Node.js, Express.js, Zod validation
- PostgreSQL, Drizzle ORM & migrations
- Clerk authentication + custom workspace RBAC / ReBAC permission engine
- Google Gemini (assistant, document & site-photo analysis, insights)
- Stripe, Resend, Sentry, PostHog, Object Storage, OpenCDE, plugin system
- Docker, VPS, GitHub Actions

Impact:

- Consolidated the full residential construction workflow into a single AI-native platform, replacing scattered tools and manual tracking.
- Fine-grained authorization engine enforcing least-privilege access and tenant isolation across hundreds of API routes.
- AI-assisted document and site-photo analysis that surfaces risks, defects, and procurement insights faster than manual review.
- Tiered billing and plan gating supporting a scalable multi-tenant SaaS model.

EDUCATION

- **Bachelor of Computer Applications, Computer Science, RPL Maheshwari College, Indore**

Certifications

- **AWS Certified Cloud Practitioner (CLF-C02)**

I hereby declare that the above-mentioned information is true to the best of my knowledge.

Jayesh Sharma

Place/Date